

基于 Netty 与 Protobuf 的独立推送网关设计方案

针对大规模、低延迟长连接场景的工业级二进制协议落地指南

在独立推送网关的架构中，采用 **Google Protocol Buffers (Protobuf)** 代替普通的 JSON 作为前后端交互的二进制协议，是实现极致性能、低带宽占用的核心手段。尤其是在海量客户端连接（如 6 万+ 终端状态同步）和频繁通信的物联网或即时通讯场景下，Protobuf 的小体积和高编解码效率能大幅降低网关的 CPU 损耗与网络带宽成本。

一、Protobuf 协议设计 (.proto)

在长连接通道中，通常采用“**单长连接多路复用**”的设计模式。前后端所有交互的消息（心跳、连接鉴权、业务通知、状态上报）都通过一个统一的顶级包装对象（Frame）进行承载。利用 Protobuf 的 `oneof` 关键字可以实现优雅的多协议复用。

```

syntax = "proto3";

package com.gateway.push.protocol;

option java_multiple_files = true;
option java_package = "com.gateway.push.protocol";
option java_outer_classname = "PushProtocol";

// 顶级外壳帧，所有的读写都以此对象为主
message Frame {
    enum Type {
        PING = 0;
        PONG = 1;
        CONNECT = 2;
        CONNECT_ACK = 3;
        NOTIFY = 4;
        BIZ_REPORT = 5;
    }

    Type type = 1;          // 消息类型
    int64 timestamp = 2;    // 时间戳
    string sequence_id = 3;  // 消息序列号，用于全链路追踪或ACK机制

    oneof body {
        ConnectRequest connectRequest = 4;
        ConnectResponse connectResponse = 5;
        Notification notification = 6;
        ReportData reportData = 7;
    }
}

message ConnectRequest {
    string token = 1;        // 连接鉴权令牌
    string client_id = 2;    // 客户端唯一ID（或终端节点ID）
    string client_version = 3; // 客户端版本
}

message ConnectResponse {
    int32 code = 1;          // 状态码：200-成功，401-鉴权失败，500-系统错误
    string msg = 2;          // 描述信息
}

message Notification {
    string topic = 1;        // 消息主题（用于层级订阅发布）
    string biz_id = 2;       // 业务流水号
    int32 priority = 3;      // 优先级
}

```

```

    string title = 4;           // 通知标题
    bytes payload = 5;         // 核心业务数据（支持任意二进制透传）
}
|
message ReportData {
    string metric = 1;         // 上报指标名称
    bytes data = 2;           // 上报二进制载荷
}

```

二、Netty 整合 WebSocket 与 Protobuf 编解码架构

在 TCP 场景下，Netty 原生提供了 `ProtobufDecoder`。但在 `WebSocket` 协议下，数据被包装在 `WebSocketFrame` 中。Protobuf 属于二进制传输，对应的帧类型为 `BinaryWebSocketFrame`。因此，我们不能直接使用 Netty 的原生解码器，需要实现自定义的适配转换层。

1. ChannelPipeline 编解码链路设计

一个标准的基于 WebSocket + Protobuf 的 Netty 管道配置顺序如下：

顺序	组件名称	作用说明
1	<code>HttpServerCodec</code>	处理 HTTP 握手请求的编解码
2	<code>HttpObjectAggregator</code>	将 HTTP 碎片聚合成完整的满载请求（默认设为 64KB）
3	<code>WebSocketServerProtocolHandler</code>	负责 WebSocket 握手、Ping/Pong 帧处理、以及连接状态维护
4	<code>WebSocketProtobufDecoder</code>	【自定义】将 <code>BinaryWebSocketFrame</code> 转换为 Protobuf 的 <code>Frame</code>
5	<code>WebSocketProtobufEncoder</code>	【自定义】将 Protobuf 的 <code>Frame</code> 编码为 <code>BinaryWebSocketFrame</code>
6	<code>GatewayPushHandler</code>	【自定义】核心业务处理器（处理鉴权、Session映射、上下线、路由注册）

三、核心组件代码落地实现

1. 自定义解码器 (WebSocketProtobufDecoder)

为了达到资深架构级的性能指标，在解码时应尽量避免分配临时字节数组 `byte[]` 导致的内存拷贝和 GC 压力。推荐通过 `ByteBuf.nioBuffer()` 直接读取 NIO 的 `ByteBuffer`，并利用 `CodedInputStream` 进行零拷贝反序列化。

```
package com.gateway.push.codec;

import com.gateway.push.protocol.Frame;
import io.netty.buffer.ByteBuf;
import io.netty.channel.ChannelHandlerContext;
import io.netty.handler.codec.MessageToMessageDecoder;
import io.netty.handler.codec.http.websocketx.BinaryWebSocketFrame;
import com.google.protobuf.CodedInputStream;

import java.nio.ByteBuffer;
import java.util.List;

public class WebSocketProtobufDecoder extends MessageToMessageDecoder<BinaryWebSocketFrame> {
    @Override
    protected void decode(ChannelHandlerContext ctx, BinaryWebSocketFrame frame, List<Object> out) throws Exception {
        ByteBuf content = frame.content();
        if (!content.isReadable()) {
            return;
        }

        // 高性能优化：避免分配 byte[]，通过 nioBuffer 建立直接视图，降低GC压力
        ByteBuffer byteBuffer = content.nioBuffer();
        CodedInputStream input = CodedInputStream.newInstance(byteBuffer);

        // 转换为 Protobuf 顶级外壳对象
        Frame decodedMessage = Frame.parseFrom(input);
        out.add(decodedMessage);
    }
}
```

2. 自定义编码器 (WebSocketProtobufEncoder)

编码器负责将后端组装好的 Protobuf 对象转换为 WebSocket 的二进制帧。可以使用 Netty 提供的堆外内存池 `ctx.alloc().buffer()` 进行高效写入。

```

package com.gateway.push.codec;

import com.gateway.push.protocol.Frame;
import io.netty.buffer.ByteBuf;
import io.netty.channel.ChannelHandlerContext;
import io.netty.handler.codec.MessageToMessageEncoder;
import io.netty.handler.codec.http.websocketx.BinaryWebSocketFrame;

public class WebSocketProtobufEncoder extends MessageToMessageEncoder<Frame> {
    @Override
    protected void encode(ChannelHandlerContext ctx, Frame msg, List<Object> out) throws Exception {
        // 将Protobuf 序列化为字节数组
        byte[] bytes = msg.toByteArray();

        // 从内存池申请一块堆外 ByteBuf
        ByteBuf byteBuf = ctx.alloc().buffer(bytes.length);
        byteBuf.writeBytes(bytes);

        // 包装为 WebSocket 专用的二进制帧输出
        out.add(new BinaryWebSocketFrame(byteBuf));
    }
}

```

3. 核心业务处理器 (GatewayPushHandler)

在此处理器中，我们将实现三大核心逻辑：**连接鉴权**、**本地会话映射**、**双向心跳维护**。这也是网关保持“有状态”长连接的核心生命线。

```

package com.gateway.push.handler;

import com.gateway.push.protocol.Frame;
import com.gateway.push.protocol.ConnectResponse;
import io.netty.channel.ChannelHandlerContext;
import io.netty.channel.SimpleChannelInboundHandler;
import java.util.concurrent.ConcurrentHashMap;

public class GatewayPushHandler extends SimpleChannelInboundHandler<Frame> {

    // 内存中的有状态会话映射表：ClientId -> Netty Channel
    private static final ConcurrentHashMap<String, ChannelHandlerContext> sessionMap = new Conc
    // 逆向映射，便于断线时清理
    private static final ConcurrentHashMap<ChannelHandlerContext, String> channelToClientMap =

    @Override
    protected void channelRead0(ChannelHandlerContext ctx, Frame frame) throws Exception {
        switch (frame.getType()) {
            case PING:
                handlePing(ctx, frame);
                break;
            case CONNECT:
                handleConnect(ctx, frame);
                break;
            case BIZ_REPORT:
                handleBizReport(ctx, frame);
                break;
            default:
                // 其他未知或不合法的中间处理
                break;
        }
    }

    private void handlePing(ChannelHandlerContext ctx, Frame frame) {
        // 收到客户端心跳，立即响应 PONG
        Frame pong = Frame.newBuilder()
            .setType(Frame.Type.PONG)
            .setTimestamp(System.currentTimeMillis())
            .setSequenceId(frame.getSequenceId())
            .build();
        ctx.writeAndFlush(pong);

        // 进阶拓展：此处还应异步刷新该客户端在 Redis 中注册的分布式路由表的 TTL
    }

    private void handleConnect(ChannelHandlerContext ctx, Frame frame) {
        var req = frame.getConnectRequest();
    }

```

```

        String token = req.getToken();
        String clientId = req.getClientId();

        // 1. 鉴权校验 (实际生产中可调用鉴权服务或校验JWT)
        boolean isAuthenticated = validateToken(token);

        Frame.Builder responseBuilder = Frame.newBuilder()
            .setType(Frame.Type.CONNECT_ACK)
            .setSequenceId(frame.getSequenceId())
            .setTimestamp(System.currentTimeMillis());

        if (isAuthenticated) {
            // 2. 本地内存映射绑定
            sessionMap.put(clientId, ctx);
            channelToClientMap.put(ctx, clientId);

            // 3. 注册到 Redis 跨节点全局路由表 (伪逻辑)
            // redisTemplate.opsForValue().set("route:" + clientId, currentServerIp, 60, TimeUnit.SECONDS);

            responseBuilder.setConnectResponse(ConnectResponse.newBuilder().setCode(200).setMsg("连接成功"));
            ctx.writeAndFlush(responseBuilder.build());
        } else {
            responseBuilder.setConnectResponse(ConnectResponse.newBuilder().setCode(401).setMsg("鉴权失败"));
            ctx.writeAndFlush(responseBuilder.build().addListener(f -> ctx.close()));
        }
    }

    private void handleBizReport(ChannelHandlerContext ctx, Frame frame) {
        // 物联网终端高频状态数据上报, 在此处直接解耦投递至 Kafka, 确保网关本身不发生计算阻塞
        String clientId = channelToClientMap.get(ctx);
        var reportData = frame.getReportData();

        // kafkaProducer.send(new ProducerRecord<>("device_metrics_topic", clientId, reportData));
    }

    private boolean validateToken(String token) {
        return token != null && !token.isBlank(); // 简化示范
    }

    @Override
    public void channelInactive(ChannelHandlerContext ctx) throws Exception {
        // 客户端掉线或主动关闭, 执行严密的资源清理逻辑
        String clientId = channelToClientMap.remove(ctx);
        if (clientId != null) {
            sessionMap.remove(clientId);
            // 同步注销 Redis 全局路由表
            // redisTemplate.delete("route:" + clientId);
        }
    }

```

```

    }
    super.channelInactive(ctx);
}

@Override
public void exceptionCaught(ChannelHandlerContext ctx, Throwable cause) {
    cause.printStackTrace(); // 生产环境中应输出结构化错误日志
    ctx.close();
}
}
}

```

💡 架构师视角：前端（Web、小程序）如何解析 Protobuf 二进制流？

前端在建立标准的 WebSocket 连接时，必须显式将 `ws.binaryType` 设置为 `"arraybuffer"`。通过引入 `protobufjs` 依赖库，前端可以将定义好的 `.proto` 文件通过工具编译为对应的 JavaScript/TypeScript 实体类文件。当 `onmessage` 触发时，直接调用 `Frame.decode(new Uint8Array(event.data))` 即可还原出与后端完全对称的结构化对象，实现真正端到端的全二进制高效交互。

四、针对大规模（6万+设备）长连接调优的关键实践

- 内存泄露防范：**Netty 的 `BinaryWebSocketFrame` 承载的底层 `ByteBuf` 采用的是引用计数机制。因为我们编写的解码器和编码器都继承自 `MessageToMessageDecoder / Encoder`，Netty 会在消息成功转换并传给下一个 Handler 后，自动负责释放原始帧的引用计数。但如果您在自定义 Handler 中使用了 `retain()` 或中断了管道流转，必须确保使用 `ReferenceCountUtil.release()`，否则会导致严重的堆外直接内存（Direct Memory）泄露。
- 空闲检测与强制剔除：**在 Pipeline 的最前端放置 Netty 自带的 `IdleStateHandler(90, 0, 0, TimeUnit.SECONDS)`。配合业务处理器，如果客户端在 90 秒内没有发送任何 PING 包，直接断开该连接（心跳超时）。这能快速清理由于弱网、基站断电造成的僵尸连接，释放操作系统的文件描述符。
- Linux 内核网络参数优化：**单机承载 6 万+ 长连接，必须在宿主机上修改 `/etc/sysctl.conf`。重点调整 `fs.file-max`（最大文件句柄数）、`net.ipv4.tcp_rmem` 和 `net.ipv4.tcp_wmem`（收发缓冲区大小，适当调小可以节省整体内存占用），并将应用程序的端口范围 `net.ipv4.ip_local_port_range` 调大，防止网关向内部转发或反向代理时端口耗尽。